

Programmera mera i Python, 7.5 hp

Provmoment: Tentamen, 5 hp.

Ladokkod A266TG

Datum och tid:

Måndag 18/3 kl.13.00 till tisdag 19/3 kl. 12.59

Plats:

Hemtentamen.

Poäng: Maximal poäng är 60 p. För godkänt krävs 36 p (60%).

Allmänna anvisningar:

- Koden ska vara lättläst och välkommenterad.
- Variabler ska ha meningsfulla namn.
- Du får inte lämna in kod som genererar felmeddelande. Då blir deluppgiften omedelbart underkänd (0 p).
- Du får inte lämna in bortkommenterad kod.
- *Varje uppgift skall lämnas in som .py-filer med följande namn:*

Uppgift_1.py (uppgift 1)

Uppgift_2.py (uppgift 2)

Uppgift_3.py (uppgift 3)

Lämna INTE in dessa filer komprimerade till en zip-fil eller i något annat komprimeringsformat.

Tentamen ska genomföras enskilt, dvs. samarbete är inte tillåtet. Det är inte heller tillåtet att använda någon form av AI-hjälpmedel, typ ChatGPT eller liknande. Genom att lämna in tentamen intygar du på heder och samvete att du har löst uppgifterna självständigt utan hjälp av någon annan person och inte heller använt någon form av AI-hjälpmedel.

Ansvariga lärare:

Hoshang Heydari, 0763-09 07 22 (uppgift 1)

Hoshang Heydari, 0763-09 07 22 (uppgift 2)

Peter Axelberg, 0760-50 70 34 (uppgift 3)

Lycka till!

Uppgift 1 (Totalt 20 p)

Ansvarig lärare:
Hoshang Heydari

- a) Skapa följande Numpy-array genom att använda tillämpa funktionen $f(x,y)=x^2+y^2$ och döp den till `NP_A`. Använd sedan denna Numpy-array för att lösa nedanstående deluppgifter. **(3p)**

```
[[ 0  1  4  9 16 25 36 49]
 [ 1  2  5 10 17 26 37 50]
 [ 4  5  8 13 20 29 40 53]
 [ 9 10 13 18 25 34 45 58]
 [16 17 20 25 32 41 52 65]
 [25 26 29 34 41 50 61 74]
 [36 37 40 45 52 61 72 85]
 [49 50 53 58 65 74 85 98]
 [64 65 68 73 80 89 100 113]
 [81 82 85 90 97 106 117 130]]
```

- b) Extrahera

```
[[ 5 20]
 [26 41]]
```

ur `NP_A` genom att använda metoden slicing och skriv ut resultatet. **(2p)**

- c) Omvandla `NP_A` till en endimensionell lista och skriv ut denna lista samt skriv ut alla element i listan som är större än 20 men mindre än 50. **(4p)**
- d) Summera det största värdet och det minsta värdet på varje rad hos `NP_A` och skriv ut resultatet på skärmen. **(3p)**
- e) Tillämpa funktionen `where()` på `NP_A` för att byta ut alla element i `NP_A` som är större än 107 till talet -100. Skriv därefter ut den modifierade Numpy-arrayen. **(2p)**
- f) Använd NumPy för att skapa nedanstående Numpy-array (endimensionell NumPy-array) och döp den till `NP_Arr` och skriv ut resultatet. Du får alltså inte kopiera och direkt använda nedanstående array, utan den ska skapas genom kodning. **(3p)**

```
[ 0.  0.  0.  0.  0.  1.  1.  1.  1.  1.  4.  4.  4.  4.
  4. 112. 112. 112. 112. 112. 16. 16. 16. 16. 16. 25. 25. 25.
 25. 25. 36. 36. 36. 36. 36. 112. 112. 112. 112. 112. 64. 64.
 64. 64. 64. 81. 81. 81. 81. 81.]
```

- g) Utgå ifrån `NP_arr` i föregående uppgift och rita ett linjediagram över polynomet $NP_Arr^{**2} + 10*NP_Arr + 5$. Använd modulen `matplotlib.pyplot` för att rita linjediagrammet. Diagrammet ska innehålla titeln 'Graf över polynom', axeletiketter med innehållet 'Polynomvärden' samt ett rutnät. OBS! Om du inte har utfört uppg. f) får du kopiera `NP_Arr` från föregående uppgift. **(3p)**

Uppgift 2 (Totalt 20 p)

Ansvarig lärare:
Hoshang Heydari

a) I den här uppgiften ska du skapa ett program som håller reda på vilka datorer som finns i ett (fiktivt) företag. Du ska lösa uppgiften genom att använda dina kunskaper i objektorienterad programmering som du fått i denna kurs. Skapa klassen *Dator* som ska hålla reda på datortillverkare och viss annan information om datorn (se nedan). Därefter ska du skriva ett huvudprogram med två egendefinierade funktioner *lista()* och *visa_data()*. Funktionen *lista()* frågar först efter antalet datorer som ska lagras och därefter matas informationen in enligt nedan. Anropar man funktionen *visa_data()* ska programmet skriva en tabell med utseende enligt nedan över de datorer som skapats med funktionen *lista()*. **(8p)**

```
Ange antal datorer som ska skrivas in:1
Ange tillverkare: HP
Ange modell: OptiPlex 7000
Ange processortyp: Core i7
Ange installerad RAM (GB): 16
Ange inköpspris: 14990
```

Tillverkare	Modell	Processortyp	Installerat RAM	Pris[kr]
HP	OptiPlex 7000	Core i7	16GB	14990

b) Du ska nu vidareutveckla programmet i uppg 2a genom att skapa underklassen *Laptop* till klassen *Dator* som sparar information om laptopdatorer. Skriv ett huvudprogram där man skapar en underklass *Laptop* till klassen *Dator* enligt följande syntaxexempel:
`Laptop('ASUS', 'ExpertBook', 'Core i5', '16', '7990', '15.6')` och därefter skriver ut informationen på skärmen: **(6p)**

```
Tillverkare: ASUS
Modell: ExpertBook
Processortyp: Core i5
Installerat RAM: 16 GB
Pris: 7990.0 Kr
Skärmstorlek: 15.6 tum
```

c) I denna deluppgift ska du använda modulen *Pickle* och skriva ett program som sparar informationen om datorerna som du skapade i uppgift 2a och 2b i en fil med namnet *dator.dat*. Du kan till exempel använda metoden *dump()* i *Pickle* för att konvertera en Python-objekthierarki till en byteström (vilket kallas serilaisering) som sedan kan sparas i en fil. Du kan skapa datafilen *dator.dat* och skriva dina datordata i den. Programrets utskrift ska ha följande utseende: **(3p)**

```
Ange datorns fabrikat: Lenovo
Ange modell: ThinkCentre M70t Tower G3
Ange processortyp: Core i7
Ange installerat RAM (GB): 16
Ange pris (kr): 9995.0

Vill du ange flera datorer? (ja/nej): nej
Data är skriven i filen dator.dat.
```

d) Nu ska vi läsa det data som du har sparat i filen *dator.dat*. Du ska göra det med hjälp av modulen *Pickle* och dess *load()*-metod. Skapa en funktion *visa_data(dator)* som skriver ut innehållet som du har sparat under uppgift 2c i på skärmen. Utskriften ska se ut enligt nedan: **(3p)**

```
Tillverkare: Lenovo  
Modell: ThinkCentre M70t Tower G3  
Processortyp: Core i7  
Installerad RAM: 16 GB  
Pris: 9995.0 Kr
```

Uppgift 3 (Totalt 20 p)

Ansvarig lärare:
Peter Axelberg

I denna inlämningsuppgift ska du med hjälp av modulerna *pandas* och *matplotlib.pyplot* analysera och vidareförädla data från amerikanska Central Intelligence Agency (CIA).

CIA tillhandahåller en referensdatabas - CIA World Factbook (<https://www.cia.gov/the-world-factbook/>) - där det finns en mängd information som fritt kan användas.

Nedan följer en kort beskrivning över innehållet i filen, som kommer att användas i denna uppgift. Ladda gärna in csv-filen i Excel eller ett motsvarande program och bekanta dig med innehållet innan du börjar att lösa uppgifterna.

Programmet du skapar ska innehålla en enkel felkontroll så att programmet inte kraschar vid otillåten inmatning.

cia_factbook.csv:

Denna csv-fil innehåller information från World Factbook om olika länders befolkning, area, ekonomi, geografi, internationella relationer med mera.

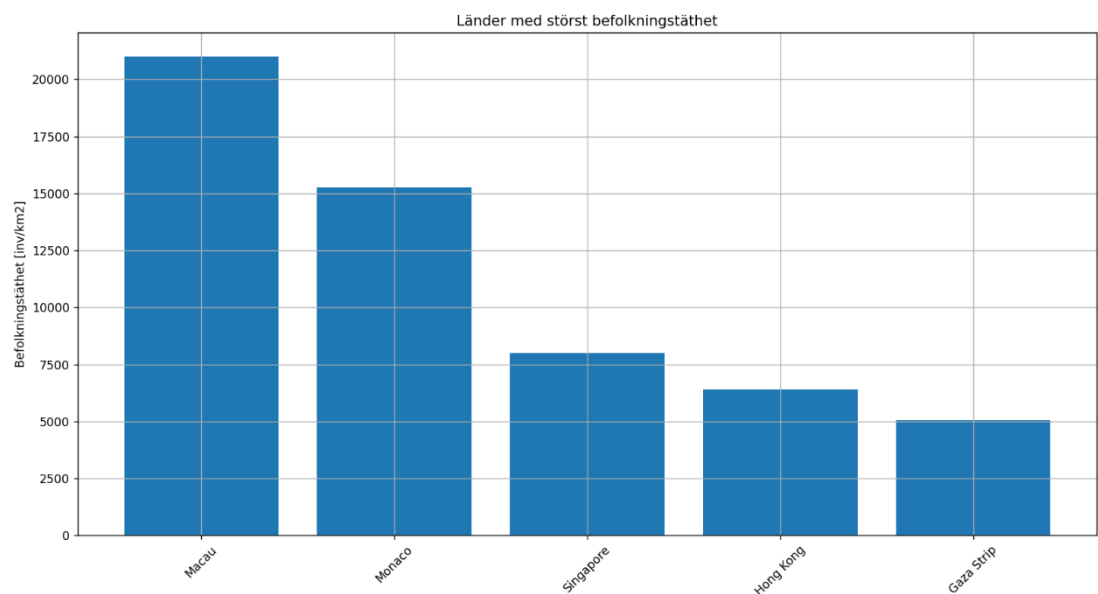
	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	Index	country	area	birth_rate	death_rate	infant_mortality	internet_usage	life_expectancy	maternal_mortality	net_migration	population	population_growth_rate		
2	1	Russia	17098242	11.87	13.83	7.08	40853000	70.16	34	1.69	1.42E+08	-0.03		
3	2	Canada	9984670	10.29	8.31	4.71	26960000	81.67	12	5.66	34834841	0.76		
4	3	United States	9826675	13.42	8.15	6.17	2.45E+08	79.56	21	2.45	3.19E+08	0.77		
5	4	China	9596960	12.17	7.44	14.79	3.89E+08	75.15	37	-0.32	1.36E+09	0.44		
6	5	Brazil	8514877	14.72	6.54	19.21	75982000	73.28	56	-0.15	2.03E+08	0.8		
7	6	Australia	7741220	12.19	7.07	4.43	15810000	82.07	7	5.74	22507617	1.09		
8	7	India	3287263	19.89	7.35	43.19	61338000	67.8	200	-0.05	1.24E+09	1.25		
9	8	Argentina	2780400	16.88	7.34	9.96	13694000	77.51	77	0	43024374	0.95		
10	9	Kazakhstan	2724900	19.61	8.31	21.61	5299000	70.24	51	0.42	17948816	1.17		
11	10	Algeria	2381741	23.99	4.31	21.76	NA	76.39	97	-0.93	38813722	1.88		
12	11	Congo, Democratic Republic of the	2344858	35.62	10.3	73.15	290000	56.54	540	-0.33	77433744	2.5		
13	12	Greenland	2166086	14.53	8.38	9.42	36000	71.82	NA	-5.98	57728	0.02		
14	13	Saudi Arabia	2149690	18.78	3.32	14.58	9774000	74.82	24	-0.59	27345986	1.49		

- a) Skapa DataFrame-objekten `df_cia_factbook` som du sedan använder när du löser nedanstående uppgifter. Obs! Separationstecknet i csv-filen är semikolletecknet (;). **(3p)**

- b) I denna uppgift ska du visa att du kan vidareförädla och presentera information som finns i *df_cia_factbook*.

Uppgiften går ut på att du ska bestämma befolkningstätheten för ett antal länder och plotta resultatet i ett stapeldiagram. Programmet ska ha en inmatningsrad där man anger antalet länder som ska presenteras i stapeldiagrammet (max 10 länder), alternativt så ska man kunna ange ett enskilt land som ska presenteras. Matar man in 7+ ska de sju länderna med störst befolkningstäthet plottas i stapeldiagrammet, matar man in 5- ska de fem länderna med lägst befolkningstäthet plottas osv. Programmet behöver inte kunna presentera länder som ligger utanför de tio med högst respektive lägst befolkningstäthet.

Anger man ett enskilt land, t.ex. Sweden (alltså ett land som står angivet i kolumnen *country*) ska landets befolkningstäthet skrivas ut på skärmen. I stapeldiagrammet ska namnet på länderna stå angivna utmed x-axeln och befolkningstätheten (antal invånare/km²) på y-axeln. Befolkningstätheten som ska presenteras ska läsas från kolumnen *density* som skapas som en ny kolumn i *df_cia_factbook* som kvoten mellan kolumnen *population* och kolumnen *area*. Tyvärr så förekommer det både oändligheten (inf) och NaN i denna kolumn. Vi löser detta genom att vi bortser från de (få) länder där man inte får ett numeriskt värde för *density* genom att stryka dessa länder ur *df_cia_factbook*. **(7p)**



- c) I denna uppgift ska du fortsätta att extrahera, vidareförädla och presentera information som finns lagrad i *df_cia_factbook* enligt nedanstående beskrivningar. Programmet ska bestå av en meny där man väljer ett av nedanstående fyra menyalternativ med passande beskrivningar på menyalternativen (du får själv tänka ut lämpliga beskrivningar på dessa). När menyalternativet är utfört ska menyn åter komma upp och ett nytt alternativ ska kunna väljas osv. Menyalternativ 4 avslutar programmet. **(10p)**

Menyalternativ 1:

I detta menyalternativ ska en tabell skrivas ut på skärmen som består av kolumnerna *country*, *area*, *birth_rate* och *life_exp_at_birth* för det land (de länder) och som uppfyller följande kriterier:

- befolkningen ska vara större än medelvärdet av befolkningen över alla länder.
- *arean* ska vara mindre än medelarean över alla länder.
- *birth_rate* (som är angivet i csv-filen som personer per 1000 invånare) ska ligga mellan 15 och 24.
- *life_exp_at_birth* ska vara högre än 70 år.

(Länder som har NaN i någon av kolumnerna i tabellen ska inte tas med i tabellen)

Land	Area [km2]	Antal födselar [per 1000 inv]	Livslängd [år]

Vietnam	331210	16	72
Philippines	2970000	21	74

Menyalternativ 2:

I detta menyalternativ ska du undersöka internetmognaden hos befolkningen i de länder som finns i *df_cia_factbook*. Gör så här:

Skapa en ny kolumn *internet_user_density* som innehåller antalet internetanvändare per 100.000 invånare. Skapa därefter en tabell som består av kolumnerna *country*, *population* och *internet_user_density* och som innehåller de fem länder med lägst antal internetanvändare per 100.000 invånare och de fem länder med flest antal internetanvändare per 100.000 invånare. OBS! I nedanstående tabell visas bara 4 länder med högst antal internetanvändare.

(Länder som har NaN i kolumnen *internet_user_density* ska inte visas i tabellen)

Land	Folkmängd	Antal internetanvändare [per 100.000 inv]

Länder med lägst antal internetanvändare per 100.000 invånare:		
Timor-Leste	1201542	174.8
Burma	55746253	197.3
Sierra Leone	5743725	259.4
Bangladesh	166280712	371.2
Congo, Democratic Republic of the	77433744	374.5
Länder med högst antal internetanvändare per 100.000 invånare:		
Iceland	317351	95036.7
Netherlands	16877351	88118.1
Saint Lucia	163362	87474.4
Sweden	9723809	86365.3

Menyalternativ 3:

I detta menyalternativ ska vi göra en uppskattning om länderna i *df_cia_factbook* har en positiv eller negativ befolkningstrend uttryckt som procent av folkmängden.

Netto befolkningsförändring per 1000 invånare får vi genom beräkningen

$population_growth_rate = (birth_rate - death_rate + net_migration_rate)$.

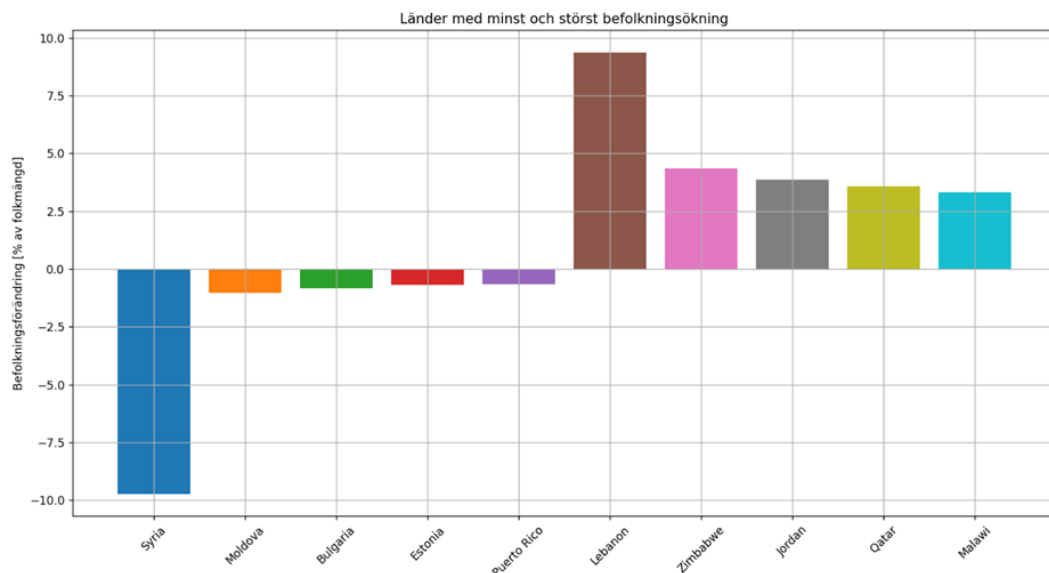
Använd sedan *population_growth_rate* för att beräkna befolkningsförändringen uttryckt i procent av folkmängden. Om resultatet av beräkningen är ett positivt tal kan vi anta en ökande befolkning annars en minskande.

Presentationen av ovanstående beräkningar ska utgöras av en tabell som innehåller de fem länder med mest negativ befolkningstrend och de fem länder med mest positiv befolkningstrend (alltså totalt 10 länder i tabellen – tabellen nedan visar bara 7 länder).

Tabellen ska innehålla kolumnerna *country*, *birth_rate*, *death_rate*, *net_migration_rate* och *population_change*. De tio länderna ska också presenteras i ett stapeldiagram där namnet på länderna finns på x-axeln och befolkningsförändringen uttryckt i procent, finns på y-axeln.

(Länder som har NaN i kolumnen *population_change* ska inte visas.)

Land	Antal födslar [per 1000 inv]	Antal döda [per 1000 inv]	Netto migration [per 1000 inv]	Befolkningsförändring [% av folkmängd]
Länder där befolkningen minskar mest :				
Syria	22	6	-113	-9.73
Moldova	12	12	-9	-1.02
Bulgaria	8	14	-2	-0.83
Estonia	10	13	-3	-0.68
Puerto Rico	10	8	-8	-0.65
Länder där befolkningen ökar mest:				
Lebanon	14	4	83	9.37
Zimbabwe	32	10	21	4.36



Menyalternativ 4:

Programmet avslutas.